Chapter 9. Recurrent Neural Networks and LSTM

Lectures 10/28, 11/2: RNNs, backpropagation through time, vanishing and exploding gradients, LSTM, language modeling, image captioning (naive)

Contents

1 Why Recurrent Networks **2**

2 The Recurrence **2**

3 Backpropagation Through Time **3**

3.1 The idea . . . 3

3.2 A worked two-step example . . . 3

3.3 General formulation . . . 4

4 Vanishing and Exploding Gradients **5**

4.1 RNNs are very deep networks . . . 5

4.2 Exploding gradients and gradient clipping . . . 5

4.3 Vanishing gradients . . . 6

5 Long Short-Term Memory **6**

5.1 The LSTM cell . . . 6

5.2 Gradient flow through the cell . . . 7

6 Applications **8**

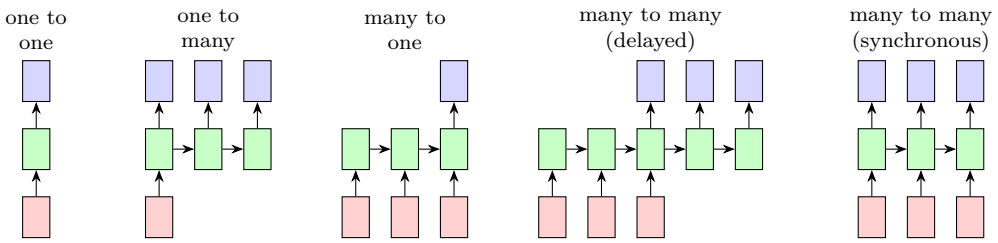
6.1 Language modeling . . . 8

6.2 Image captioning . . . 9

1 Why Recurrent Networks

A feedforward network maps one fixed-size input to one fixed-size output. Many tasks instead consume or produce **sequences**, often of variable length. A recurrent network handles them by reading one element at a time and carrying an **internal (hidden) state** that is updated as the sequence is processed. The input/output patterns seen in lecture:

Pattern	Input → output	Examples
one to one	single → single	vanilla NN / FFN
one to many	single → sequence	image to video, image captioning
many to one	sequence → single	video classification, action recognition
many to many (delayed)	read all, then emit	translation (English to Korean), video to caption
many to many (synchronous)	emit at every step	video segmentation, video editing, style transfer



Inputs (red), hidden states (green) and outputs (blue); horizontal arrows carry the state.

In the delayed pattern the outputs start only after the whole input has been read (e.g. inputs at $t = 1, 2, 3$, outputs at $t = 4, 5, 6$).

2 The Recurrence

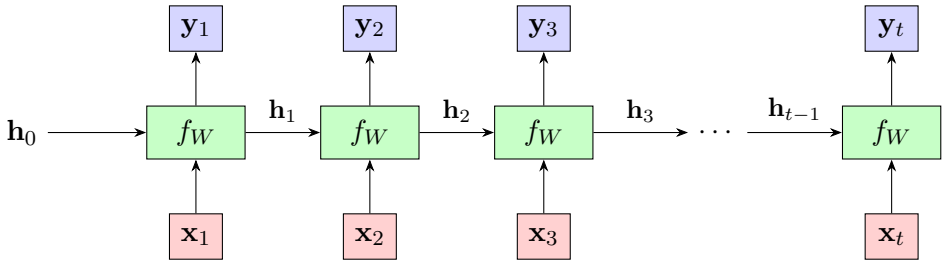
Definition 9.1: recurrent neural network

Given inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots$, an RNN computes hidden states and outputs by

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t), \qquad \mathbf{y}_t = f_{W_{hy}}(\mathbf{h}_t), \qquad t \geq 1,$$

starting from an initial state $\mathbf{h}_0$ (usually $\mathbf{h}_0 = \mathbf{0}$). The functions f_W (new state from old state and current input) and $f_{W_{hy}}$ (output from state) and their parameters are **the same at every time step**.

Unrolling the loop over time gives an ordinary computational graph with one copy of the cell per time step, all copies sharing parameters. Weight sharing is what lets one network process sequences of any length.



The unrolled RNN: every copy of the cell uses the same f_W, and the output map $f_{W_{hy}}$ is shared as well.

Definition 9.2: vanilla RNN

With an elementwise activation σ (typically tanh),

$$\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = W^{\text{out}}\mathbf{h}_t, \qquad \mathbf{h}_0 = \mathbf{0}.$$

Written out for the first two steps:

$$\mathbf{h}_1 = \sigma(W\mathbf{x}_1 + \mathbf{b}), \quad \hat{\mathbf{y}}_1 = W^{\text{out}}\mathbf{h}_1; \qquad \mathbf{h}_2 = \sigma(U\mathbf{h}_1 + W\mathbf{x}_2 + \mathbf{b}), \quad \hat{\mathbf{y}}_2 = W^{\text{out}}\mathbf{h}_2.$$

The same U, W, $\mathbf{b}$, W^{out} appear at both steps. The letters for the two weight matrices are not standardized: the next section (following the handwritten lecture notes) writes W for the recurrent weight and U for the input weight.

3 Backpropagation Through Time

3.1 The idea

The loss of a sequence is a sum of per-step losses, $L = \sum_{t=1}^{\tau} L_t$. Each L_t depends on the parameters through the current step and, via the hidden state, through every earlier step. **Backpropagation through time (BPTT)** is ordinary backpropagation on the unrolled graph. One forward pass runs through the entire sequence to compute L; one backward pass runs through the entire sequence to compute the gradient. Because parameters are shared, the gradient with respect to a parameter is the sum of the contributions from all of its copies.

Remark 9.3: truncated BPTT

For long sequences the full backward pass is expensive. **Truncated BPTT** carries the hidden state forward through the whole sequence but backpropagates only through a window of the most recent steps. Gradients from beyond the window are ignored.

3.2 A worked two-step example

Consider the scalar RNN

$$h_t = \tanh(Wh_{t-1} + Ux_t + b), \qquad Y_t = Vh_t + c, \qquad \theta = (W, U, V, b, c),$$

with inputs $(x_1, \dots, x_\tau)$, outputs $(Y_1, \dots, Y_\tau)$, targets $(Z_1, \dots, Z_\tau)$ and loss $L = \sum_t L_t$ with $L_t = \frac{1}{2}(Y_t - Z_t)^2$. To minimize L we need $\nabla_\theta L$. The chain rule now acts both within each step and across time, which is what makes this harder than for an MLP. Take $\tau = 2$ and all quantities one-dimensional. The forward pass is

$$\begin{aligned} a_1 &= Wh_0 + Ux_1 + b, \quad h_1 = \tanh a_1, \quad Y_1 = Vh_1 + c, \\ a_2 &= Wh_1 + Ux_2 + b, \quad h_2 = \tanh a_2, \quad Y_2 = Vh_2 + c, \end{aligned} \qquad L = \tfrac{1}{2}(Y_1 - Z_1)^2 + \tfrac{1}{2}(Y_2 - Z_2)^2.$$

Local derivatives. Using $\tanh'(a) = \operatorname{sech}^2 a = 1 - \tanh^2 a$, and writing ∂ for the derivative with respect to the explicit occurrence (other arguments held fixed):

$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}, \qquad \frac{\partial h_t}{\partial U} = (1 - h_t^2)x_t, \qquad \frac{\partial h_t}{\partial b} = 1 - h_t^2, \qquad \frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W.$$

Since L_t depends on h_t only through Y_t,

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial Y_t}\frac{\partial Y_t}{\partial h_t} = (Y_t - Z_t)V.$$

Output parameters. L_t depends on V and c only through Y_t, so

$$\frac{\partial L}{\partial V} = \sum_{t=1}^{2} (Y_t - Z_t) h_t, \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{2} (Y_t - Z_t).$$

Recurrent weight. L_1 sees W through h_1. L_2 sees W directly through h_2 and again through h_1, which feeds h_2:

$$\frac{\partial L}{\partial W} = \frac{\partial L_1}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W},$$

that is,

$$\frac{\partial L}{\partial W} = (Y_1 - Z_1)V(1-h_1^2)h_0 + (Y_2 - Z_2)V(1-h_2^2)h_1 + (Y_2 - Z_2)V(1-h_2^2)W(1-h_1^2)h_0.$$

Input weight and bias [Supplement]. They have the same three-path structure, with h_{t-1} replaced by x_t and by 1 respectively:

$$\frac{\partial L}{\partial U} = (Y_1 - Z_1)V(1-h_1^2)x_1 + (Y_2 - Z_2)V(1-h_2^2)x_2 + (Y_2 - Z_2)V(1-h_2^2)W(1-h_1^2)x_1,$$
$$\frac{\partial L}{\partial b} = (Y_1 - Z_1)V(1-h_1^2) + (Y_2 - Z_2)V(1-h_2^2) + (Y_2 - Z_2)V(1-h_2^2)W(1-h_1^2).$$

With $h_0 = 0$ the first and third terms of $\partial L/\partial W$ vanish, but the other two gradients keep all three terms.

3.3 General formulation

Let the input be a sequence $\tau(X) = (u_1, \dots, u_L)$ with $u_\ell \in \mathbb{R}^m$. The state update is a map $q_{\tilde\theta} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, e.g. $q_{\tilde\theta}(h, u) = \phi(Wh + Uu + c)$ with $\tilde\theta = (W, U, c) \in \mathbb{R}^p$:

$$h_0 = \mathbf{0}, \qquad h_\ell = q_{\tilde\theta}(h_{\ell-1}, u_\ell) \quad (\ell = 1, \dots, L), \qquad f_\theta(X) = Ah_L + b,$$

with $A \in \mathbb{R}^{1\times n}$, $b \in \mathbb{R}$, $h_\ell \in \mathbb{R}^n$ and $\theta = (A, b, \tilde\theta)$. For a loss $\mathcal{L}(f_\theta(X), Y)$,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial f}(f_\theta(X), Y)\,\frac{\partial f_\theta(X)}{\partial \theta}, \qquad \frac{\partial f_\theta(X)}{\partial A} = h_L^\top, \quad \frac{\partial f_\theta(X)}{\partial b} = 1, \quad \frac{\partial f_\theta(X)}{\partial \tilde\theta} = A\frac{dh_L}{d\tilde\theta}.$$

Here $dh_L/d\tilde\theta$ is the total derivative, while $\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell)$ below holds $h_{\ell-1}$ fixed. Write $J_k = \partial_h q_{\tilde\theta}(h_{k-1}, u_k) \in \mathbb{R}^{n\times n}$ for the state-to-state Jacobian.

Theorem 9.4: BPTT formula

For $\ell \le L$,

$$\frac{\partial h_L}{\partial h_\ell} = J_L J_{L-1} \cdots J_{\ell+1}, \qquad \frac{dh_L}{d\tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell),$$

(the empty product for $\ell = L$ is the identity). Consequently

$$\frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial \tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell), \qquad \frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L}\frac{\partial h_L}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} J_L \cdots J_{\ell+1}.$$

Proof. Step 1 (state Jacobian). For $k > \ell$, h_k depends on h_ℓ only through h_{k-1}, so $\partial h_k/\partial h_\ell = J_k\,\partial h_{k-1}/\partial h_\ell$. Starting from $\partial h_\ell/\partial h_\ell = I$ and iterating gives the product.

Step 2 (one-step recursion). h_ℓ depends on $\tilde\theta$ directly and through $h_{\ell-1}$, so the chain rule gives

$$\frac{dh_\ell}{d\tilde\theta} = \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell) + J_\ell \frac{dh_{\ell-1}}{d\tilde\theta}, \qquad \frac{dh_0}{d\tilde\theta} = 0.$$

Step 3 (unrolling). Induction on L. For $L = 1$ the formula reads $dh_1/d\tilde{\theta} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_0, u_1)$, which is Step 2. If the formula holds for $L-1$, Step 2 and Step 1 give

$$\frac{dh_L}{d\tilde{\theta}} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{L-1}, u_L) + \sum_{\ell=1}^{L-1} J_L \frac{\partial h_{L-1}}{\partial h_\ell} \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell) = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell} \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell).$$

Step 4 (loss). $\mathcal{L}$ depends on $\tilde{\theta}$ only through h_L. Multiplying by $\partial\mathcal{L}/\partial h_L$ and using $\partial\mathcal{L}/\partial h_\ell = (\partial\mathcal{L}/\partial h_L)(\partial h_L/\partial h_\ell)$ gives the last two identities. ■

The sum over ℓ is the parameter-sharing sum: each time step contributes the local derivative of its own copy of q, weighted by how strongly h_ℓ influences the loss.

4 Vanishing and Exploding Gradients

4.1 RNNs are very deep networks

Sequence lengths of hundreds or thousands are common, so the factor $\partial h_L/\partial h_\ell = J_L \cdots J_{\ell+1}$ in Theorem 9.4 is a product of up to $L-\ell$ Jacobians. Multiplying many numbers is unstable. If most factors exceed 1 the product blows up (**exploding gradients**). If most are below 1 it tends to 0 (**vanishing gradients**, the bigger problem). Only factors close to 1 give a product of reasonable size. For the scalar tanh RNN of Section 3.2 each factor is $\partial h_t/\partial h_{t-1} = (1 - h_t^2)W$ [Supplement].

Theorem 9.5: a sufficient condition for vanishing gradients [Supplement]

Let $q_{\tilde{\theta}}(h, u) = \tanh(Wh + Uu + c)$ and let $\|\cdot\|$ be the spectral norm. Then

$$\left\| \frac{\partial h_L}{\partial h_\ell} \right\| \le \|W\|^{L-\ell}.$$

In particular, if $\|W\| < 1$ the gradient signal from step L to step ℓ decays geometrically in $L - \ell$.

Proof. Step 1. With $a_k = Wh_{k-1} + Uu_k + c$, the Jacobian is $J_k = \operatorname{diag}(1 - \tanh^2(a_k))\, W = \operatorname{diag}(1 - h_k^2)\, W$.

Step 2. The diagonal entries lie in $[0, 1]$, so $\|\operatorname{diag}(1 - h_k^2)\| \le 1$ and $\|J_k\| \le \|W\|$.

Step 3. Submultiplicativity of the spectral norm and Theorem 9.4 give $\|J_L \cdots J_{\ell+1}\| \le \prod_{k=\ell+1}^{L} \|J_k\| \le \|W\|^{L-\ell}$. ■

The converse fails: $\|W\| > 1$ does not force explosion, because saturated units ($h_k^2 \approx 1$) shrink the factors. Exploding gradients need $\|W\|$ large and units away from saturation. [Supplement]

4.2 Exploding gradients and gradient clipping

A very large gradient means the output is very sensitive to small parameter changes in some direction. Sometimes such gradients are unworkable and the architecture must be changed. Sometimes, however, the direction is fine and only the length is the problem. Then one uses **gradient clipping** with a scalar threshold $v > 0$:

$$g \leftarrow \min\left(1, \frac{v}{\|g\|}\right) g = \begin{cases} g, & \|g\| \le v, \\ \dfrac{v}{\|g\|}\, g, & \text{otherwise.} \end{cases}$$

The clipped vector has the same direction as g and length $\min(\|g\|, v)$. The slide writes max in the first expression; the case split shows that min is meant. On a loss surface with a steep cliff, an unclipped step can jump far away from the valley, while a clipped step stays close.

4.3 Vanishing gradients

In an RNN $\partial\mathcal{L}/\partial h_L$ may be of reasonable size while

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L}\frac{\partial h_L}{\partial h_{L-1}}\frac{\partial h_{L-1}}{\partial h_{L-2}}\cdots\frac{\partial h_{\ell+1}}{\partial h_\ell}$$

is tiny. Then small changes of h_ℓ barely affect $\mathcal{L}$. Since

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = \frac{\partial \mathcal{L}}{\partial h_\ell}\frac{\partial h_\ell}{\partial u_\ell},$$

small changes of the input u_ℓ barely affect $\mathcal{L}$ either: loosely, the network does not remember u_ℓ at step L. (This is not exact, since large changes of u_ℓ may still matter.) In any case the gradient signal from time L is lost by time ℓ, so the model cannot learn which information to keep at time ℓ.

5 Long Short-Term Memory

Motivation. Plain RNNs cannot learn long-range dependencies because gradients vanish over many time steps. The LSTM adds a memory cell updated additively through gates, so gradients can flow across long time spans.

5.1 The LSTM cell

The vanilla RNN computes $h_t = \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$. The LSTM keeps a second state, the **cell state** c_t (long memory), next to the hidden state h_t, and controls it with gates.

Definition 9.6: LSTM

With σ the logistic sigmoid and $\odot$ elementwise multiplication,

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

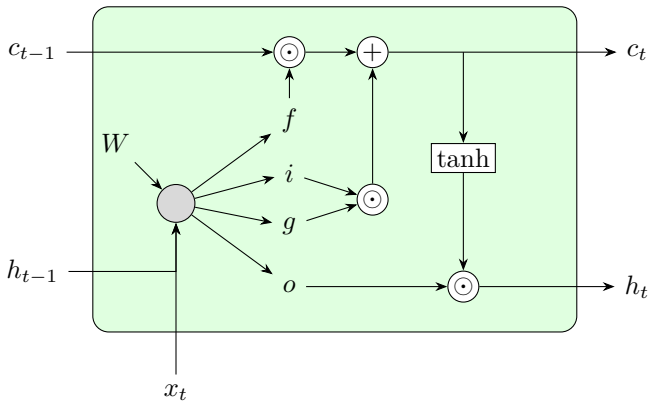
The four gates have the following roles.

- Input gate i_t: how much of the new content g_t to write into the cell.
- Forget gate f_t: how much of the old cell c_{t-1} to carry over.
- Output gate o_t: how much of the cell to reveal in h_t.
- Candidate ("gate gate") g_t: the new content computed from the current input.

Stacking the four pre-activations gives the compact form

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix},$$

with one matrix W of size $4h \times 2h$ when the hidden state and the input both have dimension h (in general $4h \times (h+m)$ for input dimension m). Its output has length $4h$ and is split into four blocks of length h. Another common notation concatenates $[h_{t-1}, x_t]$, writes $\tilde{C}_t$ for g_t and $*$ for $\odot$.



The LSTM cell: W acts on the stacked (h_{t-1}, x_t); the cell state runs along the top.

5.2 Gradient flow through the cell

Theorem 9.7: the cell path

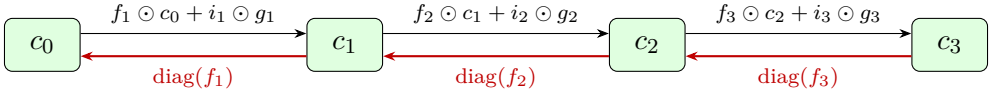
Holding the gates fixed, $\dfrac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t)$. Along the path that goes only through cell states,

$$\frac{\partial c_L}{\partial c_\ell} = \operatorname{diag}\Big(\prod_{k=\ell+1}^{L} f_k\Big) \qquad \text{(product taken entrywise).}$$

Proof. Step 1. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ the second term does not contain c_{t-1}, and the first is the diagonal linear map $c_{t-1} \mapsto \operatorname{diag}(f_t) c_{t-1}$.

Step 2. Chaining these Jacobians and multiplying diagonal matrices entrywise gives the product formula. ■

Compare the vanilla RNN, where each factor is $\operatorname{diag}(1 - h_t^2)W$: the same matrix W is multiplied in at every step and the tanh derivative can only shrink it. In the LSTM the factor is the forget gate itself, chosen by the network at each step, with no repeated matrix product and no squashing derivative. If $f_t \approx 1$ the gradient passes through many steps almost unchanged ("uninterrupted gradient flow"). For example, with $f = 1$ and $i = 0$ the content of the cell is preserved indefinitely. For a vanilla RNN it is much harder to learn a recurrent matrix W that preserves information in the hidden state.



Forward (black) and backward (red) along the cell states: the backward signal is only multiplied by forget gates.

Remark 9.8: what the LSTM does not guarantee

The full Jacobian $\partial c_L / \partial c_\ell$ also has paths through h_{t-1} and the gates, and f_t may be far from 1. So the LSTM does not rule out vanishing or exploding gradients. It gives the model an easier way to learn long-range dependencies.

6 Applications

The lecture covered two applications: language modeling with RNNs, and image captioning. As a further example it mentioned SIRD epidemic models. There the dynamics are an ODE whose infection rate α changes over time, and an LSTM can learn the sequence of α from data.

6.1 Language modeling

A sentence is a sequence of discrete symbols (tokens). Sentence generation, machine translation and text summarization are all instances of modeling language with a network f_θ.

Definition 9.9: one-hot encoding (tokenization)

With a vocabulary of n tokens, token t is represented by $\mathbf{x}_t \in \mathbb{R}^n$ having a single 1 at the index of that token and zeros elsewhere.

Definition 9.10: RNN language model

Sentence generation is next-token prediction. At step t:

$$\begin{aligned} \mathbf{z}_t &= W_e \mathbf{x}_t + \mathbf{b}_e, & W_e &\in \mathbb{R}^{d\times n}, \\ \mathbf{h}_t &= \mathrm{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \theta), & \mathbf{h}_t &\in \mathbb{R}^d, \\ \hat{\mathbf{x}}_{t+1} &= W_d \mathbf{h}_t + \mathbf{b}_d, & W_d &\in \mathbb{R}^{n\times d}. \end{aligned}$$

The three lines are the continuous word embedding, the recurrent network (whose state is the continuous output embedding), and the classification layer giving scores over the vocabulary. Generation starts from the start-of-sentence token `<SOS>` as $\mathbf{x}_0$ and ends at the end-of-sentence token `<EOS>`.

Training. The model reads the true sentence shifted by one, `<SOS>`, $\mathbf{x}_1, \ldots, \mathbf{x}_{T-1}$, and at each step predicts the next token. The loss is a word-level classification loss,

$$\sum_{t=1}^{T} \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t), \qquad \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t) = -\log \mathrm{softmax}(\hat{\mathbf{x}}_t)_{k_t},$$

where k_t is the index of the true token. The model's sentence probability factors as

$$p_\theta(\mathbf{x}_1, \ldots, \mathbf{x}_T) = \prod_{t=1}^{T} p_\theta(\mathbf{x}_t \mid \mathbf{x}_0, \ldots, \mathbf{x}_{t-1}).$$

So minimizing the summed cross-entropy is maximum likelihood for this probability (Chapter 2).

Inference. At each step take the output scores $S(\mathbf{x}_t)$ and choose a token by one of two rules.

- Greedy (deterministic): $\mathbf{x}_t = \arg\max S(\mathbf{x}_t)$.
- Probabilistic (stochastic): sample $\mathbf{x}_t \sim S(\mathbf{x}_t)$ from the softmax probabilities.

Feed the chosen token back as the next input, and stop when `<EOS>` is sampled. Greedy decoding always returns the same sentence. Sampling can also pick less likely words, so repeated runs give different sentences.

Machine translation reads a sentence in one language and generates its translation in another. This is the delayed many-to-many pattern.

6.2 Image captioning

The task is to describe an image $\mathbf{x}$ with a sentence $\mathbf{y}$. It splits into two steps.

1. Recognize the visual patterns in the image with a CNN g_ϕ (Chapter 7).
2. Generate a sentence conditioned on the image with an RNN language model.

The model is

$$f_\theta(\mathbf{y} \mid g_\phi(\mathbf{x})) :$$

the image features $g_\phi(\mathbf{x})$ condition the language model that produces $\mathbf{y}$, e.g. "man in black shirt is playing guitar." The lecture listed naive captioning and captioning with attention as the two variants. Attention-based captioning opens Chapter 10.

Naive captioning. Take a CNN pretrained on ImageNet and keep the feature vector of its last fully connected layer (dimension 4096 for VGG, 2048 for ResNet) for a $224 \times 224 \times 3$ input. A linear layer maps this vector to the word-embedding dimension, and the result is fed to the LSTM language model as if it were the first word. Then `<sos>` is fed, and generation continues as in Definition 9.10 ("Man", "in", ..., "guitar", `<eos>`). Training uses (image, sentence) pairs and the same word-level cross-entropy loss as sentence generation.

A practical issue. A strong language model can produce plausible sentences from the previous words alone, ignoring the image. The previous words are at dependency distance 1 from the next word, whereas the image enters only at the first step and must be carried through the whole hidden-state chain. So the conditioning on the image has to be strengthened.

- **Shortcuts.** Feed the image feature at every step as an extra input. This reduces the dependency distance to the image to 1, but the model can still learn to ignore it.
- **Attention.** Let the model look at different image regions for different words (Chapter 10). Using the same weight for every region (uniform attention) gives back a single global feature.